

LAB # 03a

Construct of Escape sequence, variables, various data types and in java.

Objective

To get familiarity with the escape sequence, variables and various data types in java.

Lab Goals

To provide working insight into the usage of various java programming tools:

- a. Demonstrate the working of class.
- b. Understand escape sequences (\n, \t, \", \\\, etc.) in Java for manipulating output.
- c. Learn naming conventions for identifiers in Java.
- d. Declare and initialize variables of different data types.
- e. Understand numerical data types (int, long, short, byte) and their ranges.
- f. Explore floating-point data types (float, double) and their precision.
- g. Understand character, string, and Boolean data types in Java.

CLASS

A class is a user defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one.

Create java class

You can use the New Java Class wizard to create a Java class. The Java Class wizard can be invoked in different ways –

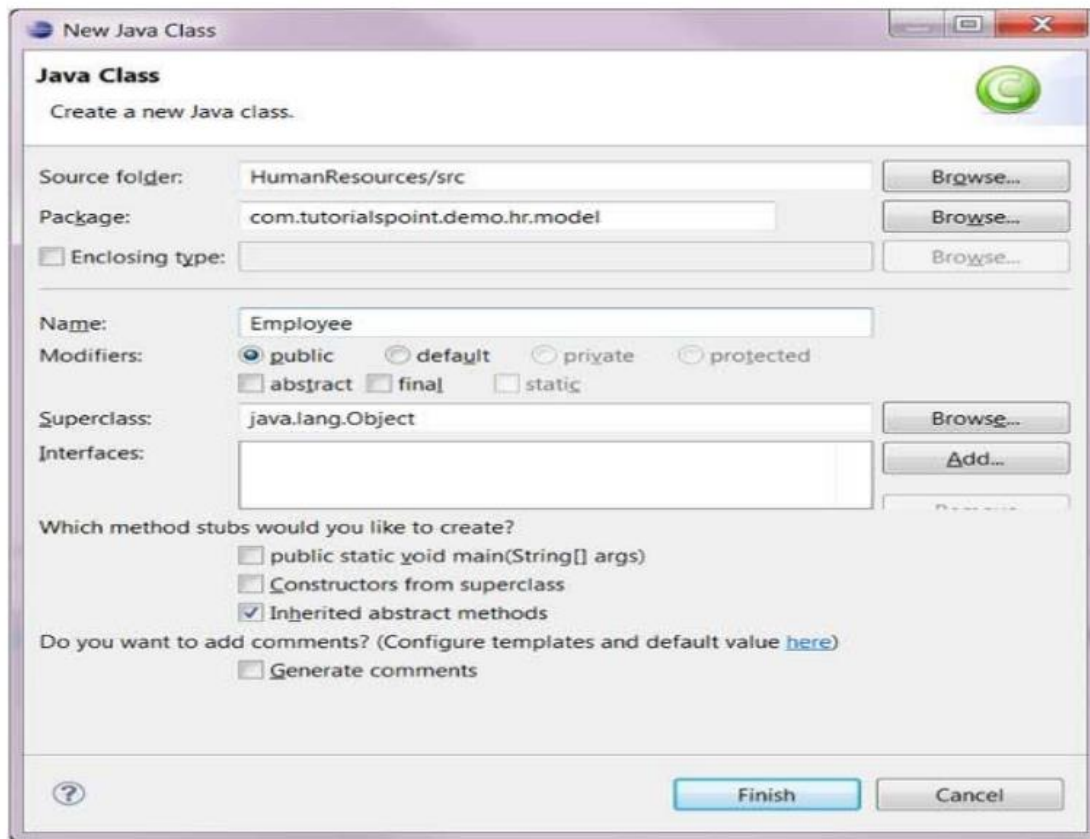
- By clicking on the File menu and selecting New → Class.
- By right clicking in the package explorer and selecting New → Class.
- By clicking on the class drop down button () and selecting class ().

Before bringing up the New Java Class wizard, if possible, select the package in which the class is to be created so that the wizard can automatically fill in the package name for you.

USING THE NEW JAVA CLASS WIZARD

Once the java class wizard comes up –

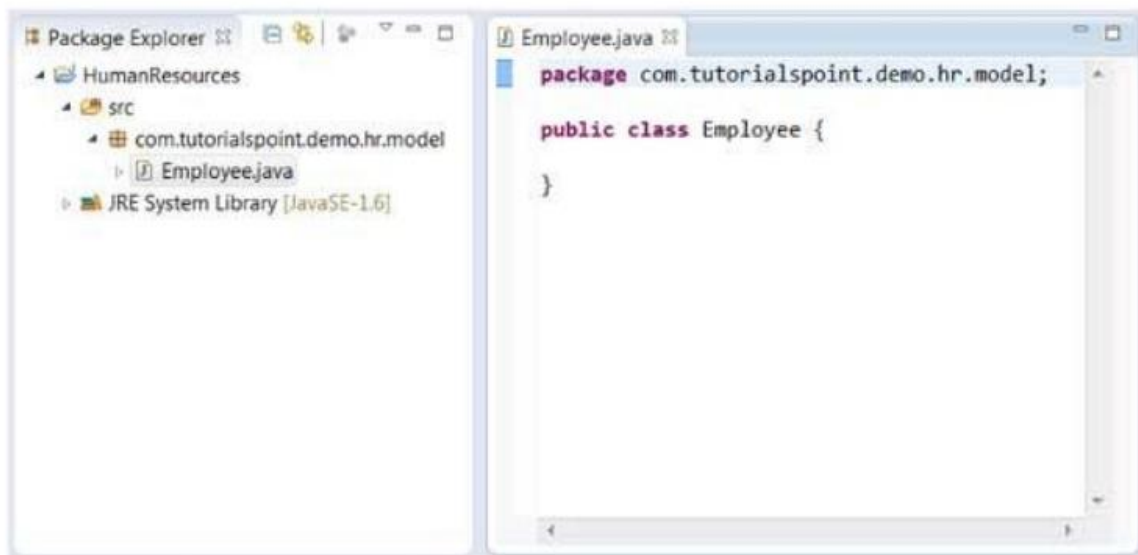
- Ensure the source folder and package are correct.
- Enter the class name.
- Select the appropriate class modifier.
- Enter the super class name or click on the Browse button to search for an existing class.
- Click on the Add button to select the interfaces implemented by this class.
- Examine and modify the check boxes related to method stubs and comment



- Click the Finish button.

USING THE NEW JAVA CLASS WIZARD

The newly created class should appear in the Package Explorer view and a java editor instance that allows you to modify the new class. It should appear in the editor area.



ESCAPE SEQUENCE IN JAVA:

A character with a backslash (\) just before it is an escape sequence or escape character. We use escape characters to perform some specific task. The total number of escape sequences or escape characters in Java is 8. Each escape character is a valid character literal. The list of Java escape sequences:

Escape sequence	Meaning
\\	Backslash
\a	Alert
\b	Backspace
\'	Single quotation mark
\''	Double quotation mark
\n	New line
\t	Tab
\r	Carriage return
\f	Formfeed
\d	Octal
\xd	Hexadecimal

What is the need to use Escape Sequence in Java?

Suppose we want to display a message with quotation marks in the output. Can we write a statement to print a message on the console like this:

```
System.out.println("He said "Java is fun unlimited");
```

No, we cannot write this statement to print message because the compiler will generate compile-time error. Java compiler will think that the second quotation character is the end of the string and does not know what to do with the rest of the characters. To overcome it, Java introduces a special notation to represent special characters. This special notation is called escape sequence. It consists of a backslash (\) followed by a character or a combination of digits.

For example, \t is an escape sequence for the Tab character. \u0009 is a Unicode to represent Tab character. \" is an escape sequence for double quotes. \u0022 is Unicode for it.

Let's take an example program where we will print the quoted message using escape sequence.

Example Program 1:

```
public class A {  
    public static void main(String[] args) {  
  
        System.out.println("He said \"Java is fun unlimited\"");  
    }  
}
```

Output:

```
He said "Java is fun unlimited"
```

Note: The symbols \ and " together represent a single character.

NAMING CONVENTIONS

Naming convention basically describe that how you can assign name to your identifiers such as class, package, variable, constant, method etc. It is highly recommended for developer/programmers to follow naming convention while writing code. All the classes, interfaces, packages, methods and fields of java programming language are given according to java naming convention.

Naming convention for different identifiers in JAVA:

Class Name: Should start with uppercase letter and be a noun e.g. String, Color, Button, System, Thread etc.

Example:

```
Class Student{  
    int rollNumber,marks;  
    String name;  
}
```

Interface Name: Should start with uppercase letter and be an adjective e.g. Runnable, Remote, ActionListener etc.

Example:

```
public interface ItemClickListener {  
    void onItemClick(View view, int position);  
}
```

Method Name: Should start with lowercase letter and be a verb e.g. actionPerformed(), main(), print(), println() etc.

Example:

```
private void addNumbers(){  
    int firstNumber=10,secondNumber=20;  
    System.out.println(firstNumber+secondNumber);  
}
```

Variable Name: Should start with lowercase letter e.g. firstName, secondName etc

Example:

```
private void addNumbers(){  
    int firstNumber=10,secondNumber=20;  
    System.out.println(firstNumber+secondNumber);  
}
```

Package Name: Should be in lowercase letter e.g. java, lang, sql, util etc.

Example:

```
package com.demo;
```

Constant Name: Should be in uppercase letter e.g. RED, YELLOW, MAX_PRIORITY

Example:

```
final int MAX_PRIORITY=5;
final String ACCOUNT_TYPE="saving";
```

ADVANTAGES OF NAMING CONVENTION:

1. By using standard Java naming conventions, you make your code **easier** to read for yourself and for other programmers.
2. Its increase the **readability** of program.
3. It will take **less time** to figure out what the code does.

VARIABLES

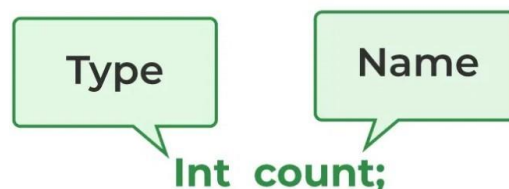
Variables are containers for storing data values. Variables in Java language are the names used to refer to data stored in the memory. One Java variable can hold only a single type of data. It is a combination of "vary + able" which means its value can be changed.

Rules to Declare a Variable

1. A variable name can consist of Capital letters **A-Z**, lowercase letters **a-z** digits **0-9**, and two special characters such as **_** underscore and **\$** dollar sign.
2. The first character must not be a digit.
3. Blank spaces cannot be used in variable names.
4. Java keywords cannot be used as variable names.
5. Variable names are case-sensitive.
6. There is no limit on the length of a variable name but by convention, it should be between 4 to 15 chars.

How to Declare Variables in Java?

We can declare variables in Java as pictorially depicted below as a visual aid.



From the image, it can be easily perceived that while declaring a variable, we need to take care of two things that are:

1. **datatype:** Type of data that can be stored in this variable.
2. **data_name:** Name was given to the variable.

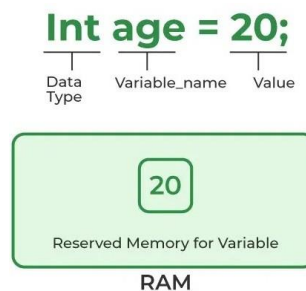
In this way, a name can only be given to a memory location. It can be assigned values in two ways:

- Variable Initialization
- Assigning value by taking input

How to Initialize Variables in Java?

It can be perceived with the help of 3 components that are as follows:

- **datatype:** Type of data that can be stored in this variable.
- **variable_name:** Name given to the variable.
- **value:** It is the initial value stored in the variable.



TYPES OF VARIABLES IN JAVA

1. Local Variables
2. Instance Variables
3. Static Variables

1) Local Variable

A variable declared inside the body of the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

A local variable cannot be defined with "static" keyword.

```
public void displayMessage() {  
    String message = "Hello, World!"; // Local variable  
    System.out.println(message);  
}
```

2) Instance Variable

A variable declared inside the class but outside the body of the method, is called an instance variable. It is not declared as static.

It is called an instance variable because its value is instance-specific and is not shared among instances.

```
public class Student {  
    String name; // Instance variable  
    public void setName(String newName) {  
        name = newName;  
    }  
}
```

3) Static variable

A variable that is declared as static is called a static variable. It cannot be local. You can create a single copy of the static variable and share it among all the instances of the class. Memory allocation for static variables happens only once when the class is loaded in the memory.

```
public class Counter {  
    static int count = 0; // Static variable  
    public Counter() {  
        count++;  
    }  
}
```

Data Types in Java

Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:

1. **Primitive data types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
2. **Non-primitive data types:** The non-primitive data types include Classes, Interfaces, and Arrays.

There are 8 types of primitive data types:

1. boolean
2. byte
3. char
4. short
5. int
6. long
7. float
8. double

Data Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
short	2 bytes	Stores whole numbers from -32,768 to 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
boolean	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter or ASCII values

Boolean Data Type

The Boolean data type is used to store only two possible values: true and false. This data type is used for simple flags that track true/false conditions. The Boolean data type specifies one bit of information, but its "size" can't be defined precisely.

Example: Boolean one = false

Byte Data Type

The byte data type is an example of primitive data type. It is an 8-bit signed two's complement integer. Its value-range lies between -128 to 127 (inclusive). Its minimum value is -128 and maximum value is 127. Its default value is 0. The byte data type is used to save memory in large arrays where the memory savings is most required. It saves space because a byte is 4 times smaller than an integer. It can also be used in place of "int" data type.

Example: byte a = 10, byte b = -20

Short Data Type

The short data type is a 16-bit signed two's complement integer. Its value-range lies between -32,768 to 32,767 (inclusive). Its minimum value is -32,768 and maximum value is 32,767. Its default value is 0. The short data type can also be used to save memory just like byte data type. A short data type is 2 times smaller than an integer.

Example: short s = 10000, short r = -5000

Int Data Type

The int data type is a 32-bit signed two's complement integer. Its value-range lies between - 2,147,483,648 (-2^{31}) to 2,147,483,647 ($2^{31} - 1$) (inclusive). Its minimum value is - 2,147,483,648 and maximum value is 2,147,483,647. Its default value is 0. The int data type is generally used as a default data type for integral values unless if there is no problem about memory.

Example: int a = 100000, int b = -200000

Long Data Type

The long data type is a 64-bit two's complement integer. Its value-range lies between -9,223,372,036,854,775,808 (-2^{63}) to 9,223,372,036,854,775,807 ($2^{63} - 1$) (inclusive). Its minimum value is - 9,223,372,036,854,775,808 and maximum value is 9,223,372,036,854,775,807. Its default value is 0. The long data type is used when you need a range of values more than those provided by int.

Example: long a = 100000L, long b = -200000L

Float Data Type

The float data type is a single-precision 32-bit IEEE 754 floating point. Its value range is unlimited. It is recommended to use a float (instead of double) if you need to save memory in large arrays of floating point numbers. The float data type should never be used for precise values, such as currency. Its default value is 0.0F.

Example: float f1 = 234.5f

Double Data Type

The double data type is a double-precision 64-bit IEEE 754 floating point. Its value range is unlimited. The double data type is generally used for decimal values just like float. The double data type also should never be used for precise values, such as currency. Its default value is 0.0d.

Example: double d1 = 12.3

Char Data Type

The char data type is a single 16-bit Unicode character. Its value-range lies between '\u0000' (or 0) to '\uffff' (or 65,535 inclusive). The char data type is used to store characters.

Example: char letterA = 'A'

Create the following Java programs in your editor of choice and save it as “<classname>.java”

```
class Demo {  
    public static void main(String args[]) {  
        byte x;  
        int a = 270;  
        double b = 128.128;  
        System.out.println(a);  
        System.out.println("a and x " + a + " " + x);  
        System.out.println(b);  
        System.out.println("b and a " + b + " " + a);  
        System.out.println("b and x " + b + " " + x);  
    }  
}
```

Exercise

A. Write a Java program that:

- Declares and initializes a char variable for gender (M or F)
- Declares and initializes a boolean variable for driving license status (true or false)
- Displays the following output using escape sequences:

Expected Output:

A screenshot of an IDE's output window titled "Output - SampleTest (run)". It shows the execution of a Java program. The output text is: "run: Gender: M Has driving license: true BUILD SUCCESSFUL (total time: 0 seconds)". The window has standard IDE icons on the left (run, debug, etc.) and a close button on the right.

B. Write a Java program that:

- Take two String variables for first name and last name as user input.
- Concatenates them to display the full name in the following format using escape sequences:

Expected Output:

A screenshot of an IDE's output window titled "Output - SampleTest (run)". It shows the execution of a Java program. The output text is: "run: Full Name: \"John Doe\" BUILD SUCCESSFUL (total time: 0 seconds)". The window has standard IDE icons on the left and a close button on the right.

C. Write a Java program to create a store billing system that:

- Takes the names of 5 items and their respective prices as input (prices should be of type double).
- Calculates the total bill.
- Type casts the total bill into an int and displays both the original and type-casted bill.

Expected Output:

```
Enter the name of item 1: Apple
Enter the price of Apple: 2.5
Enter the name of item 2: Milk
Enter the price of Milk: 1.75
Enter the name of item 3: Bread
Enter the price of Bread: 1.25
Enter the name of item 4: Cheese
Enter the price of Cheese: 3.50
Enter the name of item 5: Juice
Enter the price of Juice: 2.99

Total Bill (Original): $11.99
Total Bill (Rounded): $11
```

D. Write a Java program to create an employee salary management system that:

1. **Takes the following inputs from the user:**
 - Employee Name (String)
 - Employee ID (int)
 - Basic Salary (double)
 - Allowance (float)
 - Tax Rate (in percentage as int)
2. **Calculates the following:**
 - Gross Salary = Basic Salary + Allowance
 - Tax Amount = (Gross Salary * Tax Rate) / 100
 - Net Salary = Gross Salary - Tax Amount
3. **Type cast the following:**
 - Convert tax rate to **double** for calculation
 - Convert net salary to **int** for display purposes

Expected Output:

```
Employee Name: John Doe
Employee ID: 101
Basic Salary: $5000.50
Allowance: $200.75
Gross Salary: $5201.25
Tax Amount (15%): $780.18
Net Salary (Original): $4421.07
Net Salary (Rounded): $4421
```

E. Write a Java program that:

1. **Takes the following inputs from the user:**

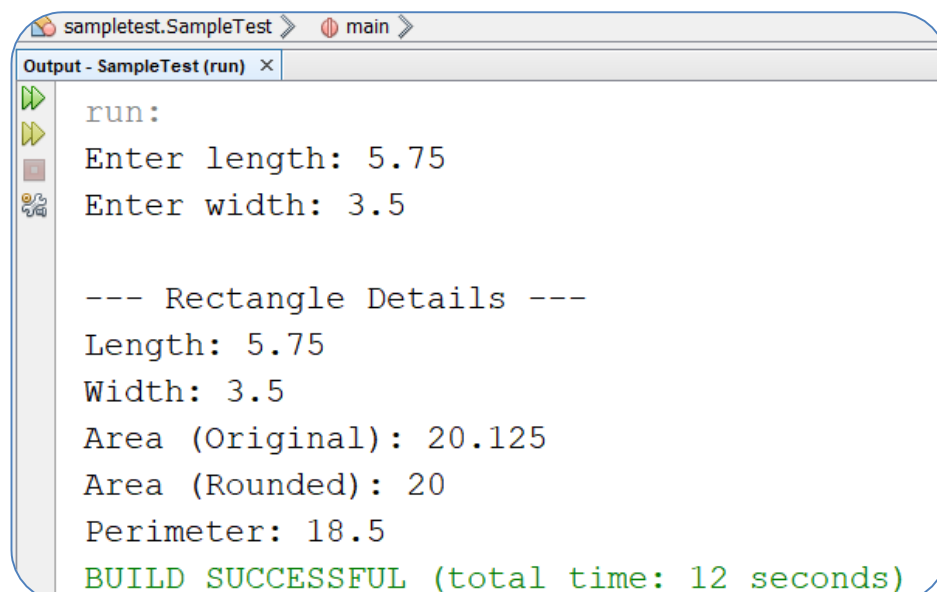
- Length (double)
- Width (double)

2. **Calculates:**

- $\text{Area} = \text{Length} \times \text{Width}$
- $\text{Perimeter} = 2 \times (\text{Length} + \text{Width})$

3. Type cast the **area** to an **int** and display it.

Expected Output:



```
sampletest.SampleTest > main >
Output - SampleTest (run) x
run:
Enter length: 5.75
Enter width: 3.5

--- Rectangle Details ---
Length: 5.75
Width: 3.5
Area (Original): 20.125
Area (Rounded): 20
Perimeter: 18.5
BUILD SUCCESSFUL (total time: 12 seconds)
```